Problem ID	Captured Flag	Steps (Summary)
P1 – dossier_101	`fsuCTF{1_kn0w_n0th1n3_4b0u7_f1nl4n6}`	Extract VBA from .docm, decode Base64+XOR to find URL, download PowerShell payload, decrypt AES-CBC with correct key/IV/ciphertext mapping
P2 – Schrodinger	`fsuCTF{d0_n07_ob53rv3}`	Extract red and blue channel LSBs via zsteg, interleave characters to reconstruct flag
P3 – The Orb	`fsuCTF{Magic_Orb}`	Repair PCAP header corruption, reconstruct embedded ELF, fix e_phoff and INTERP vaddr, execute binary to print flag
P4 – Fix-It-Felix	`fsuCTF{3v3ry0n3_63t5_C4k3}`	Fix 7z StartHeaderCRC (0xffffffff → 0x1324b0f7), decrypt AES-256 with password from filename, extract Flag.png

P1 – dossier_101

This challenge provided a .docm. The file contained obfuscated VBA macros that acted as a staged malware dropper, ultimately leading to an AES-encrypted PowerShell payload containing the flag.

Initial File Identification & Extraction

The .docm format is a ZIP container with embedded VBA macros. Unzipping reveals the internal structure including word/vbaProject.bin where the macros live.

```
$ file dossier_101.docm
dossier_101.docm: Microsoft Word 2007+

$ unzip dossier_101.docm -d docm_extracted/
$ ls docm_extracted/word/
  document.xml  vbaProject.bin  ...
```

Macro Analysis

Used olevba (oletools) to extract and analyze the VBA macros. The macro contained several suspicious behaviors:

1. Base64 decoding of an embedded blob
2. XOR operations using a hardcoded key
3. PowerShell execution via Shell()
4. Network download via MSXML2.XMLHTTP

```
$ olevba dossier_101.docm
VBA MACRO ThisDocument.cls
  Dim encoded As String
  encoded = "<base64 blob>"
  Dim key As Byte: key = 0x41
  ' XOR decode then execute via PowerShell...
```

Base64 + XOR Deobfuscation

1. Extracted the encoded blob from the macro
2. Decoded the Base64 data
3. Applied XOR using the key found in the script
4. Recovered the hidden URL: https://dacxserver.com/mortar.ps1

PowerShell Payload Analysis

Downloaded mortar.ps1 and identified AES decryption logic with heavy obfuscation. Located the key, IV, and ciphertext sources. The critical fix was correctly mapping which variable served which role:

```
Correct mapping:
  Finland substring  → ciphertext
  hex string 8329... → AES key
  hex string 1337dead... → IV
```

After correcting the variable roles and running AES-CBC decryption, the flag was recovered:

Flag: **fsuCTF{1_kn0w_n0th1n3_4b0u7_f1nl4n6}**

P2 – Schrodinger

This challenge provided a PNG image with the description 'time to brute force' and a fake flag containing a YouTube link hinting 'roses are red, violets are blue.' The real flag was hidden across two color channels using bit-plane steganography.

Discovery

Running zsteg -a on the image revealed readable strings hidden in specific bit planes:

```
$ zsteg -a Schrodinger.png
b8,r,lsb,xy  .. text: "fuT{0n7o5r3666"
b8,b,lsb,xy  .. text: "sCFd_0_b3v}"
```

The fake flag found earlier was:

```
fsuCTF{https://youtu.be/Crl4T-pZnbc}
```

The YouTube video title referenced 'roses are red, violets are blue' which was a direct hint to extract the red and blue channels.

Solution

1. Identify the fake flag via zsteg; note the YouTube link hint about roses (red) and violets (blue)
2. Extract bit plane 8, LSB, xy-order from the red channel: fuT{0n7o5r3666
3. Extract bit plane 8, LSB, xy-order from the blue channel: sCFd_0_b3v}
4. Interleave characters from red and blue channels alternately to reconstruct the flag
5. Ignore trailing characters from the longer red channel (overflow padding)

Character-by-character interleaving:

```
Red:    f  u  T  {  0  n  7  o  5  r  3  6  6  6
Blue:   s  C  F  d  _  0  _  b  3  v  }

Result: f s u C T F { d 0 _ n 0 7 _ o b 5 3 r v }
```

Flag: **fsuCTF{d0_n07_ob53rv3}**

P3 – The Orb

This challenge provided a file named 'Orb' which would not open in Wireshark. Analysis revealed it was a corrupted PCAP file with a hidden ELF binary embedded inside the packet payloads.

PCAP Repair

The file had multiple corruptions in the global PCAP header:

```
Original (corrupt):
  Version major: 0x0102  (should be 0x0002)
  Version minor: 0x0001  (should be 0x0004)
  orig_len = 0 in first 3 packet records (should match incl_len)

Fix applied:
  data[4] = 0x02  # version major = 2
  data[5] = 0x00
  data[6] = 0x04  # version minor = 4
  data[7] = 0x00
  # Set orig_len = incl_len wherever orig_len was 0
```

After repair, the PCAP opened in Wireshark and contained 5 packet records. Packets 3 and 5 contained the payload readable strings like /lib64/ld-linux-x86-64.so.2, __libc_start_main, and 'You pondered your orb and found a flag' confirmed an ELF binary was embedded.

ELF Reconstruction

The key insight was that the PCAP global header (bytes 16–23) perfectly overlapped with an ELF header's e_type, e_machine, and e_version fields:

```
PCAP snaplen field (bytes 16-19): 03 00 3e 00
  -> ELF e_type    = 0x0003 (ET_DYN / PIE)
  -> ELF e_machine = 0x003e (EM_X86_64)

PCAP linktype field (bytes 20-23): 01 00 00 00
  -> ELF e_version = 1  (correct)
```

1. Replace bytes 0–15 (PCAP magic + version) with correct ELF e_ident (7f 45 4c 46 02 01 01 00 ...)
2. Fix e_phoff: corrupted value 0x4000000040 -> corrected to 0x40
3. Fix INTERP segment p_vaddr: 0x31800000318 -> 0x318
4. Mark as executable and run

```
$ ./orb_fixed
You pondered your orb and found a flag
fsuCTF{Magic_Orb}
```

Flag: **`fsuCTF{Magic_Orb}`**

P4 – Fix-It-Felix

This challenge provided a file named 'broken', a 7-zip archive that could not be opened.

Archive Repair

The 7z global header (SignatureHeader) has a StartHeaderCRC field that covers the 20-byte StartHeader. In the broken file this field was set to 0xffffffff. This is an indicator of corruption:

```
Stored StartHeaderCRC:   0xffffffff  <- corrupted
Correct CRC32 of bytes [12:32]:  0x1324b0f7

Fix: data[8:12] = struct.pack('<I', 0x1324b0f7)

Verification:
  NextHeaderOffset = 9086  (ok)
  NextHeaderSize   = 35    (ok)
  NextHeaderCRC    = 0x726e2a10  (matched, no further corruption)
```

Decryption

The repaired archive used Method = LZMA2:12k + 7zAES encryption. Parsing the compressed end-header revealed two files inside:

```
files/Flag.png
files/therecouldnotbeabetterpassword
```

The AES-256 key derivation parameters extracted from the header:

```
NumCyclesPower = 19  (2^19 = 524,288 SHA-256 iterations)
Salt size      = 0   (no salt)
IV (16 bytes)  = 4f db 89 ff a2 be c4 55 a4 46 a4 fc d3 eb b5 41
```

The second filename, therecouldnotbeabetterpassword, is also the archive password. Decrypting it produced a valid 12,169-byte PNG. The second file's content read: "There may be a better password."

Solution Steps

5. Identify StartHeaderCRC = 0xffffffff as corrupted
6. Recompute correct CRC32 over bytes [12:32] of the header
7. Write fixed CRC back to bytes [8:12] and save repaired .7z
8. Parse the compressed end-header to find filenames and AES params
9. Use filename 'therecouldnotbeabetterpassword' as the decryption password
10. Run: 7z x fixed.7z -ptherecouldnotbeabetterpassword
11. Extract Flag.png containing the flag

```
$ 7z x fixed.7z -ptherecouldnotbeabetterpassword

Extracting archive: fixed.7z
Method = LZMA2:12k 7zAES
Everything is Ok

Extracted: files/Flag.png
Extracted: files/therecouldnotbeabetterpassword
```

Flag: **fsuCTF{3v3ry0n3_63t5_C4k3}**